CS6120 Assignment 3: Word Representation - REPORT

Deepanshu Bhagria

Part I

1.1 Data preparation. I loaded the Quotes-500K dataset from Kaggle into a pandas DataFrame with three columns: quote, author and category. It started with 499,709 rows, and after dropping rows with a missing quote or author I had 497,956.

1.2.1 Authors and preprocessing. A lot of author entries had the book title stuck on after a comma, like "Martin Luther King Jr., A Testament of Hope", so I kept only the part before the first comma. That brought the number of unique authors down from 117,296 to 77,636. For the text, I lowercased everything, removed apostrophes and punctuation, and removed stopwords using Gensim's stopword list. Then I combined all the quotes from each author into one document, which gave me 77,636 author documents.

1.2.2 TF-IDF. I picked Albert Einstein, Mark Twain and Oscar Wilde. Their two most frequent words with at least four letters were life and human for Einstein, good and thing for Twain, and life and people for Wilde. I implemented TF-IDF myself, with TF as the word count divided by the document length and IDF as log(N / number of authors who use the word), where N is all 77,636 author documents.

```
tfidf_df.pivot(index="author", columns="word", values="tfidf")
```

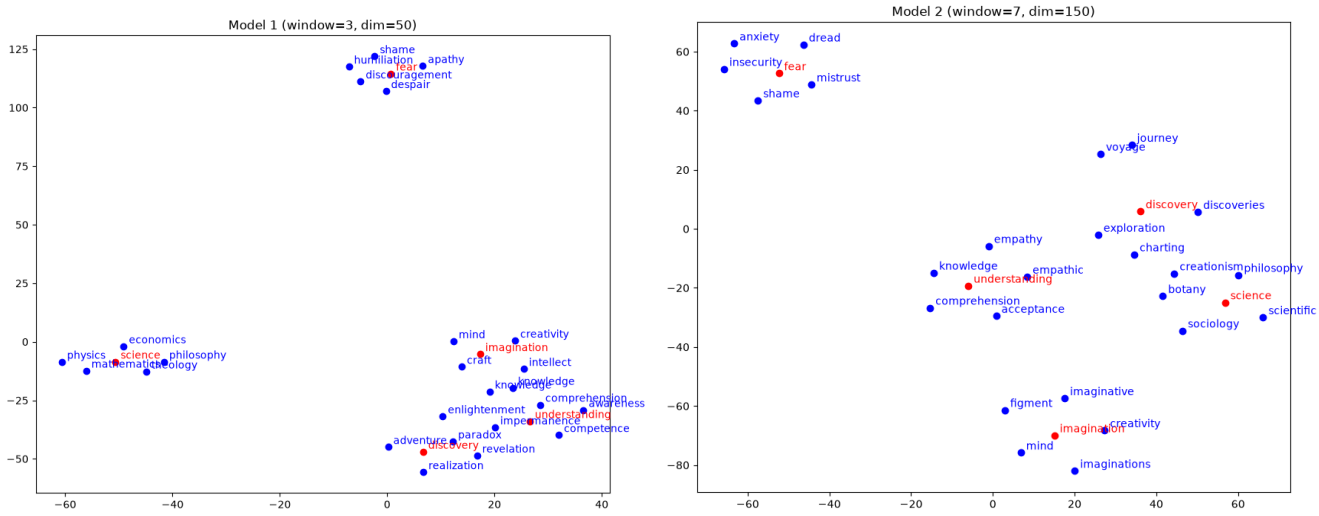
✓ 0.0s

word	good	human	life	people	thing
author					
Albert Einstein	0.004321	0.018532	0.016165	0.011237	0.009631
Mark Twain	0.014139	0.007042	0.006612	0.005793	0.014280
Oscar Wilde	0.012590	0.002636	0.020246	0.015978	0.018011

The IDF weighting changed the ranking for two of the three authors. Einstein uses "life" the most, but "human" gets a higher TF-IDF because far fewer authors use it (IDF 2.43 vs 1.43). The same thing happened with Twain, where "thing" ends up just ahead of "good" even though "good" appears more often. "Human" also separates the authors best, since it's high for Einstein and really low for Wilde, while "life" has the lowest IDF of all five words because so many authors use it.

1.2.3 Word2Vec. I trained two skip-gram models with Gensim, treating each quote as one sentence and keeping the stopwords so the context windows stay realistic. Model 1 used a window of 3, 50 dimensions and 3 negative samples, and Model 2 used a window of 7, 150 dimensions and 10 negative samples. For “imagination”, Model 1 gave words like intellect, knowledge, mind, creativity and intuition, while Model 2 gave imaginations, imaginative, figment, fantasy and overactive. The similarity between “fear” and “knowledge” was 0.491 in Model 1 and 0.377 in Model 2.

1.2.4 t-SNE. I plotted the five key words together with their five closest words for each model.



1.2.5 Comparison. In Model 1, imagination was clearly closest to understanding (0.661), but in Model 2 it was about equally close to science (0.448), understanding (0.444) and fear (0.432). With a window of 7 the model sees most of the quote, so words get tied to what they show up alongside, not just to words used the same way. The one pair that actually got closer was discovery and science (0.493 → 0.515), which became the most similar pair in Model 2 and fits the idea that a bigger window groups words by topic. You can see this in the neighbours too, since Model 2 added words like figment and overactive that usually appear right around "imagination".

As for dimensionality, almost every other similarity dropped in Model 2; for example, fear/knowledge went from 0.491 to 0.377. My guess is that 150 dimensions gives the model more room to spread words apart, while 50 dimensions squeezes abstract words together. Some things stayed the same, like fear being the least similar to science in both models (0.268 and 0.226). The two models shared anywhere from 0 (discovery) to 6 (science) of their top 10 neighbours, and since I changed the window, dimensions and negative samples all at once, I can't say exactly which setting caused which difference.

Part II

2.1 Analogy test. I compared two pretrained 300-dimensional embeddings loaded through Gensim. The first is word2vec trained on Google News (about 100 billion words, case-sensitive; Mikolov et al., 2013), where I only kept the 400,000 most common words so it would fit in memory. The second is GloVe trained on Wikipedia 2014 and Gigaword 5 (about 6 billion words, 400,000 lowercase words; Pennington et al., 2014). I implemented Equation 2 myself: compute b − a + c, find the word with the highest cosine similarity to it, and leave out a, b and c. I lowercased the questions for GloVe, and any question with a missing word counted as wrong.

	word2vec	GloVe
capital-world	0.791	0.960
currency	0.320	0.158
city-in-state	0.720	0.593
family	0.852	0.881
gram1-adjective-to-adverb	0.293	0.226
gram2-opposite	0.431	0.273
gram3-comparative	0.911	0.881
gram6-nationality-adjective	0.901	0.926
average	0.652	0.612

Both models did well on simple, regular patterns like family words, comparatives and nationalities, getting over 85% right. GloVe was much better at capital cities, word2vec did better on currencies and US cities with their states.

Both models struggled most with adjective-to-adverb and opposites. For opposites, a lot of the answers are rare words like "uncompetitive", and opposites tend to sit close together in the embedding space anyway. For adverbs, the model often picked another form of the adjective instead of the actual adverb.

2.2 Antonyms. I looked at the top 10 most similar words for increase, enter and open. For "increase", word2vec ranked "decrease" as the single most similar word (0.837), even above "increases" and "increased", and GloVe had it third. "Closed" was in the top 5 for "open" in both models. "Enter" was the exception, since "exit" didn't show up, probably because "exit" is mostly used as a noun (exit polls, exit signs).

I think this happens because these embeddings learn a word's meaning from the words around it, and opposites are used in almost the same way. You see "prices will increase by 5%" and "prices will decrease by 5%", or "the store opens at 9" and "the store closes at 9", so both words end up with nearly the same vector. They also often appear right next to each other, like "increase or decrease", which pulls them even closer. So the model learns which words are related, but not whether they mean the same thing or the opposite.

2.3 Bonus.

I tested two new analogy types: countries in conflict (India : Pakistan :: Russia : Ukraine, Greece : Turkey :: Israel : Palestine, China : Taiwan :: Armenia : Azerbaijan) and sport to league (basketball : NBA :: hockey : NHL, hockey : NHL :: baseball : MLB, baseball : MLB :: football : NFL). Both models got all three sports questions right, which makes sense because leagues are mentioned in a really consistent way in news and Wikipedia. Overall word2vec got 3 out of 6 and GloVe got 4 out of 6.

The conflict questions were much harder. GloVe got Russia → Ukraine, probably because the Crimea annexation was all over Wikipedia in 2014, while word2vec picked Azerbaijan, another former Soviet country. For Armenia, GloVe picked Nagorno-Karabakh, the region Armenia and Azerbaijan are fighting over, so it “knew” the conflict but gave the place instead of the country. For Israel and Armenia the models often just returned “Israeli” or “Armenian”, which shows that when they can’t find the relationship, they fall back to a word close to the third word.